Security-Focused Guide for AI Code Assistant Instructions

by the OpenSSF Best Practices and the AI/ML Working Groups, 2025-08-01

AI code assistants can significantly speed up development. However, they need guidance to produce **secure** and robust code. This guide explains how to improve the security of their results by creating custom prompts or custom instructions (e.g. Claude markdown, GitHub Copilot instructions file, Cline instructions file, Cursor rules, Kiro steering, etc.). These instructions ensure the AI assistant accounts for application code security, supply chain safety, and platform or language-specific considerations. They also help embed a "security conscience" into the tool. In practice, this means fewer vulnerabilities making it into your codebase. Remember that these instructions should be kept concise, specific, and actionable. The goal is to influence the AI's behaviour without overwhelming it. [wiz2025a]

These recommendations are based on expert opinion and various recommendations in the literature. We encourage experimentation and feedback to improve these recommendations. We, as an industry, are together learning how to best use these tools.

---

1. TL;DR

Short on time? Here's what really matters:

- **You Are the Developer – AI is the Assistant:** The developer (you) remains in full control of the code, and you are responsible for any harms that may be caused by the code. Critically evaluate and edit AI-generated code just as you would code written by a human colleague and never blindly accept suggestions in situations where that could eventually cause harm. [ifip2021] [anssibsi2024a]
- **Apply Engineering Best Practices Always:** AI-generated code isn't a shortcut around engineering processes such as code reviews, testing, static analysis, documentation, and version control discipline. [markvero2025a]
- **Be Security-Conscious:** Assume AI-written code can have bugs or vulnerabilities, because it often does. AI coding assistants can introduce security issues like using outdated cryptography or outdated dependencies, ignoring error handling, or leaking secrets. Check for any secrets or sensitive data in the suggested code. Make sure dependency suggestions are safe and not pulling in known vulnerable packages. [shihchiehdai2025a], [anssibsi2024b]

- **Guide the AI:** AI is a powerful assistant, but it works best with your guidance. Write clear precise prompts that specify security requirements. Don't hesitate to modify or reject AI outputs. Direct your AI tool to build its own instructions file based on this guide. [swaroopdora2025a] [haoyan2025a]
- **Ask the AI to review and improve its own work**. Once you have some AI-written code, where possible, ask it to review and improve its own work (repeating these steps as necessary). This technique is sometimes called Recursive Criticism and Improvement (RCI) and can be remarkably effective. For instance, "Review your previous answer and find problems with your answer" followed by "Based on the problems you found, improve your answer" for one or more iterations. Encourage the use of tools such as linters, SAST, dependency checkers, etc. through the improvement cycles. [catherinetony2024a]
- **Express your concerns to the AI**. If you have concerns about something AI has generated, express your concerns in detail, and ask it to analyze that code to determine whether or not it's okay. Include relevant information to increase the likelihood of a useful response. Ensure that if something is stated as a fact, it's actually a fact. Review that answer.

By keeping these points in mind, you can harness AI code assistants effectively without sacrificing quality or security.

1.1. TL;DR Sample Instructions

Here are sample instructions that you can copy and paste. In most cases you should **extract from** this sample (for details see below). If you copy and paste irrelevant parts, the AI is more likely to generate extraneous or even incorrect code as it attempts to compensate for attacks that can't happen:

> *Provide secure code. User inputs should be checked for expected format and length. Always validate function arguments and use parameterized queries for database access. Escape special characters in user-generated content before rendering it in HTML. When generating output contexts such as HTML or SQL, use safe frameworks or encoding functions to avoid vulnerabilities. Never include API keys, passwords, or secrets in code output, and use environment variables or secure vault references instead. Use secure authentication flows (for instance, using industry-standard libraries for handling passwords or tokens) and to enforce role-based access checks where appropriate. Use constant-time comparison when timing differences could leak sensitive information, such as when comparing session identifiers, API keys, authentication tokens, password hashes, or nonces. When generating code, handle errors gracefully and log them, but do not expose internal details or secrets in error messages Use logging frameworks that can be configured for security. Prefer safe defaults in configurations – for example, use HTTPS by default, require strong encryption algorithms, and disable insecure protocols or options. Follow least privilege in any configuration or code. When applicable, generate unit tests for security-critical functions (including negative tests to ensure the code fails safely). If you generate placeholder code (e.g.,* `TODO` *comments), ensure it is marked for security review before deployment. Avoid logging sensitive information of PII. Ensure that no sensitive or PII is stored in plaintext. Use popular, community-trusted libraries for common tasks (and avoid adding obscure dependencies if a standard library or well-known package can do the same job). Do not add dependencies that may be malicious or hallucinated. Always use the official package manager for the given language (npm, pip, Maven, etc.) to install libraries, rather than copying code snippets. Specify version ranges or exact versions. When suggesting dependency versions, prefer the latest stable release and mention updating dependencies regularly to patch vulnerabilities. Generate a Software Bill of Materials (SBOM) by using tools that support standard formats like SPDX or*

CycloneDX. Where applicable, use in-toto attestations or similar frameworks to create verifiable records of your build and deployment processes. Prefer high-level libraries for cryptography rather than rolling your own.

When adding important external resources (scripts, containers, etc.), include steps to verify integrity (like checksum verification or signature validation) if applicable. When writing file or OS-level operations, use safe functions and check for errors (e.g., use secure file modes, avoid temp files without proper randomness, etc.). If running as a service, drop privileges when possible. Always include appropriate security headers (Content Security Policy, X-Frame-Options, etc.) in web responses, and use frameworks' built-in protections for cookies and sessions. When generating code for cloud services (AWS/Azure/GCP), follow the provider's security guidelines (e.g., use parameterized queries for cloud databases, encrypt data at rest and in transit, handle keys via cloud KMS). When using containers, use minimal base images and avoid running containers with the root user. Use official images from trusted sources, and pin image versions using immutable digests (e.g., SHA256 hashes) instead of mutable tags like latest. When working with container images, verify both the integrity and authenticity of images using container signing tools like cosign or notation. Include steps to verify signatures from trusted publishers and implement admission controllers in Kubernetes to enforce signature verification policies. When generating HTML/JS, do not include direct links to untrusted third-party hosts for critical libraries; use our locally hosted or CDN with integrity checks. For mobile and desktop apps, do not suggest storing sensitive data in plaintext on the device; use the platform's secure storage APIs. When generating github actions or CI/CD pipelines, ensure secrets are stored securely (e.g., using GitHub Secrets or environment variables) and not hard-coded in the workflow files. Include steps to run security scans (SAST/DAST) and dependency checks in the CI/CD pipeline to catch vulnerabilities early. When generating infrastructure-as-code (IaC) scripts, ensure they follow security best practices (e.g., restrict access to resources, use secure storage for secrets, and validate inputs) and use the latest versions of devops dependencies such as GitHub actions and lock the version to specific SHA. In C or C++ code, always use bounds-checked functions (e.g., `strncpy` or `strlcpy` over `strcpy`), avoid dangerous functions like `gets`, and include buffer size constants to prevent overflow. Enable compiler defenses (stack canaries, fortify source, DEP/NX) in any build configurations you suggest. In Rust code, avoid using `unsafe` blocks unless absolutely necessary and document any `unsafe` usage with justification. In any memory-safe language, prefer using safe library functions and types; don't circumvent their safety without cause. In go code, use the data race detector when building the application. For Python, do not use `exec` / `eval` on user input and prefer safe APIs (e.g., use the `subprocess` module with `shell=False` to avoid shell injection). For Python, follow PEP 8 and use type hints, as this can catch misuse early. For JavaScript/TypeScript, when generating Node.js code, use prepared statements for database queries (just like any other language) and encode any data that goes into HTML to prevent XSS. For Java, when suggesting web code (e.g., using Spring), ensure to use built-in security annotations and avoid old, vulnerable libraries (e.g., use `BCryptPasswordEncoder` rather than writing a custom password hash). For C#, Use .NET's cryptography and identity libraries instead of custom solutions.

Never suggest turning off security features like XML entity security or type checking during deserialization. Code suggestions should adhere to OWASP Top 10 principles (e.g., avoid injection, enforce access control) and follow the OWASP ASVS requirements where applicable. Our project follows SAFECode's secure development practices – the AI should prioritize those (e.g., proper

> *validation, authentication, cryptography usage per SAFECode guidance). When generating code, consider compliance requirements (e.g., HIPAA privacy rules for medical data, PCI-DSS for credit card info) – do not output code that logs or transmits sensitive data in insecure ways. Include comments or TODOs in code suggesting security reviews for complex logic, and note if any third-party component might need a future update or audit. When writing or reviewing code, run or simulate the use of tools like CodeQL, Bandit, Semgrep, or OWASP Dependency-Check. Identify any flagged vulnerabilities or outdated dependencies and revise the code accordingly. Repeat this process until the code passes all simulated scans.*

Follow this with:

> *Review your previous answer and find problems with your answer.*

Follow this with:

> *Based on the problems you found, improve your answer.*

If you see an issue in specific results, ask something like:

> *Analyze (specific area of code) to determine if it has (kind of vulnerability). Consider (relevant information 1, 2, 3, e.g., information about the code, language, etc.). Justify your answer with specific evidence.*

---

2. Secure Coding Principles in AI Instructions

One of the first sections in your instructions should reinforce general secure coding best practices. These principles apply to all languages and frameworks, and you want the AI to **always** keep them in mind when generating code:

- **Input Validation & Output Encoding:** Instruct the AI to treat all external inputs as untrusted and to validate them. *Example: "user inputs should be checked for expected format and length"*. Any output should be properly encoded to prevent injection attacks such as SQL injection or cross-site scripting (XSS). *Example: "Always validate function arguments and use parameterized queries for database access"* and *"Escape special characters in user-generated content before rendering it in HTML"*. Similarly, specify that when generating output contexts such as HTML or SQL, the assistant should use safe frameworks or encoding functions to avoid vulnerabilities. [swaroopdora2025b] [wiz2025b] [haoyan2025b]
- **Authentication, Authorization & Secrets Management:** Emphasize that credentials and sensitive tokens must never be hard-coded or exposed, use secure authentication flows, and use constant-time comparisons when appropriate. Your instructions could say: *"Never include API keys, passwords, or secrets in code output, and use environment variables or secure vault references instead. Use secure authentication flows (for instance, using industry-standard libraries for handling passwords or tokens) and to enforce role-based access checks where appropriate. Use constant-time comparison when timing differences could leak sensitive information, such as when comparing session identifiers, API keys, authentication tokens, password hashes, or nonces."*[hammondpearce2021a] [neilperry2022a] [swaroopdora2025c]

- **Error Handling & Logging:** Guide the AI to implement errors securely by catching exceptions and failures without revealing sensitive info (stack traces, server paths, etc.) to the end-user. In your instructions, you might include: *"When generating code, handle errors gracefully and log them, but do not expose internal details or secrets in error messages".* This ensures the assistant's suggestions include secure error-handling patterns (like generic user-facing messages and detailed logs only on the server side). Additionally, instruct the AI to use logging frameworks that can be configured for security (e.g. avoiding logging of personal data or secrets). [swaroopdora2025d]
- **Secure Defaults & Configurations:** Include guidance such as: *"Prefer safe defaults in configurations – for example, use HTTPS by default, require strong encryption algorithms, and disable insecure protocols or options".* By specifying this, the AI will be more likely to generate code that opts-in to security features. Always instruct the AI to follow the principle of least privilege (e.g. minimal file system permissions, least-privileged user accounts for services, etc.) in any configuration or code it proposes. [wiz2025c] [swaroopdora2025e]
- **Testing for Security:** Encourage the AI to produce or suggest tests for critical code paths including negative tests that verify that what shouldn't happen, doesn't happen. In your instructions, add: *"When applicable, generate unit tests for security-critical functions (including negative tests to ensure the code fails safely)"*. [anssibsi2024c] [markvero2025b]
- **Data Protection:** When generating code, always prioritize data minimization and avoid storing or processing confidential or otherwise sensitive information (like personal data - PII) unless absolutely necessary. For that case, suggest strong encryption at rest and in transit, and recommend techniques like anonymization. For example: *"Generate a function that securely handles user input for a registration form, asking for necessary fields to avoid logging sensitive PII. Ensure that no sensitive or PII is stored in plaintext"*. [swaroopdora2025f]

Note that we are *not* currently recommending in the general case that the AI be told to respond from a particular viewpoint (e.g., a role or persona) or character a.k.a. "persona pattern/memetic proxy". An example of this approach would be the instruction "Act as a software security expert. Provide outputs that a security expert would give". Some experiments found that telling the system it is an expert often makes it perform poorly or worse on these tasks [catherinetony2024b] [connordilgren2015b]. However, we encourage continued experimentation, and may change our recommendations based on future information.

---

3. Addressing Software Supply Chain Security

Modern software heavily relies on third-party libraries and dependencies. It's crucial that your AI assistant's instructions cover supply chain security, ensuring that suggested dependencies and build processes are secure:

- **Safe Dependency Selection:** Instruct the AI to prefer well-vetted, reputable libraries when suggesting code that pulls in external packages. This is especially important to counter hallucinated package names; one study found 19.7% proposed packages did not exist [josephspracklen2024e]. These hallucinations enable "slopsquatting" attacks, where attackers create malicious packages with names commonly hallucinated by AI models [billtoulas2025a]. While it's important to independently check new dependencies, AI tools can often self-identify these names if asked to do so [josephspracklen2024f]. For example: *"Use popular, community-trusted libraries for common tasks (and avoid adding obscure dependencies if a standard library or well-known package can do the same job). Do not add*

dependencies that may be malicious or hallucinated.". Emphasize evaluating packages before use – as a developer would manually. [josephspracklen2024a]

- **Use Package Managers & Lock Versions:** Your instructions should tell the AI to use proper package management. For instance: *"Always use the official package manager for the given language (npm, pip, Maven, etc.) to install libraries, rather than copying code snippets"*. Also, instruct it to specify version ranges or exact versions that are known to be secure. By doing so, the AI will generate code that, for example, uses a `requirements.txt` or `package.json` entry, which aids in maintaining supply chain integrity. [openssf2023a]
- **Stay Updated & Monitor Vulnerabilities:** Include guidance for keeping dependencies up-to-date. For example: *"When suggesting dependency versions, prefer the latest stable release and mention updating dependencies regularly to patch vulnerabilities"*. [arifulhaque2025a] [josephspracklen2024b]
- **Generate Software Bill of Materials (SBOM):** Instruct the AI to create and maintaining SBOMs for better visibility into your software supply chain. For example: *"Generate an SBOM by using tools that support standard formats like SPDX or CycloneDX"*. You can also mention provenance tracking: *"Where applicable, use an attestations framework such as in-toto or similar to create verifiable records of your build and deployment processes"*. This ensures comprehensive tracking of what goes into your software and provides the foundation for ongoing vulnerability monitoring and incident response. [anssibsi2024e]
- **Integrity Verification:** To further secure the supply chain, you can instruct the assistant to show how to verify what it uses. For instance: *"When adding important external resources (scripts, containers, etc.), include steps to verify integrity (like checksum verification or signature validation) if applicable"*. [josephspracklen2024c]

---

4. Platform and Runtime Security Considerations

Every software runs somewhere – be it a web browser, a cloud platform, an IoT device, or an OS. Your guide for the AI assistant should have a section addressing platform-specific and runtime security concerns:

- **Operating System and Environment:** If your code will run on a server or user machine, instruct the AI on safe system interactions. *"When writing file or OS-level operations, use safe functions and check for errors (e.g., use secure file modes, avoid temp files without proper randomness, etc.). If running as a service, drop privileges when possible"*. This ensures the AI's output respects security at the OS level (for example, not recommending running as root unless absolutely necessary). Emphasize least privilege: for instance, code for a service should use the minimal privileges needed. The assistant could then suggest using system APIs that limit permissions or show how to configure a Linux capability instead of full root access. [arifulhaque2025b] [shihchiehdai2025b]
- **Web and Cloud Platforms:** For web applications, direct the AI to include web security best practices. *"Always include appropriate security headers (Content Security Policy, X-Frame-Options, etc.) in web responses, and use frameworks' built-in protections for cookies and sessions"*. You can also mention using HTTPS for any client-server communication. On cloud platforms, instruct it on secure use of cloud services: *"When generating code for cloud services (AWS/Azure/GCP), follow the provider's security guidelines (e.g., use parameterized queries for cloud databases, encrypt data at rest and in transit, handle keys via cloud KMS)"*. [swaroopdora2025g] [neilperry2022b]
- **Container and Deployment Considerations:** If your project uses containers or orchestrators (like Docker, Kubernetes), include guidance like: *"Use minimal base images and avoid running containers with the root user. Use official images from trusted sources, and pin image versions using immutable digests (e.g.,*

SHA256 hashes) instead of mutable tags like latest". This will cause the AI to generate Dockerfiles or deployment scripts that adhere to container security best practices and ties into supply chain concerns as well – using only trusted images and specific versions that ensure consistency and prevent supply chain attacks. Additionally, instruct the AI on container signature verification: *"When working with container images, verify both the integrity and authenticity of images using container signing tools like cosign or notation. Include steps to verify signatures from trusted publishers and implement admission controllers in Kubernetes to enforce signature verification policies".* Instruct the AI to further control deployment artifacts. For instance, by ensuring production websites only load resources from trusted domains to avoid subversion: *"When generating HTML/JS, do not include direct links to untrusted third-party hosts for critical libraries; use our locally hosted or CDN with integrity checks".* [anssibsi2024f] [josephspracklen2024d]

- **Mobile and Desktop App Security:** If relevant, instruct on platform-specific secure coding. For mobile apps, you might say: *"Do not suggest storing sensitive data in plaintext on the device; use the platform's secure storage APIs".* For desktop apps, *"Prefer high-level libraries for cryptography rather than rolling your own".* These platform notes ensure the AI isn't blind to the context in which code runs. It will include, for example, usage of Android's `SharedPreferences` with encryption or Apple's Keychain for iOS if dealing with credentials, reflecting platform best practices. [hammondpearce2021b]
- **Operational Security:** Instruct the AI to consider devops security aspects. For example: *"When generating github actions or CI/CD pipelines, ensure secrets are stored securely (e.g., using GitHub Secrets or environment variables) and not hard-coded in the workflow files"*. This will help the AI generate secure CI/CD configurations that prevent accidental exposure of sensitive information. You can also instruct it to include security checks in the pipeline: *"Include steps to run security scans (SAST/DAST) and dependency checks in the CI/CD pipeline to catch vulnerabilities early"*. This ensures that the AI's generated code integrates security into the development lifecycle, rather than treating it as an afterthought. Treat devops tools with the same security rigor as application code. For example, instruct the AI to use secure coding practices in infrastructure-as-code (IaC) scripts, such as Terraform or CloudFormation: *"When generating IaC scripts, ensure they follow security best practices (e.g., restrict access to resources, use secure storage for secrets, and validate inputs)"* and *"use the latest versions of devops dependencies such as GitHub actions and lock the version to specific SHA"*. [anssibsi2024g] [openssf2023b]

---

5. Programming Language-Specific Security Examples

It's valuable to dedicate individual paragraphs to language-specific security considerations that are relevant to your project. The following section serves as examples, or special cases the AI should handle to highlight unique vulnerabilities or features of that language and how the AI should approach them:

- **C/C++ (Memory-Unsafe Languages):** For languages without automatic memory safety, instruct the AI to be extra cautious with memory management. *"In C or C++ code, always use bounds-checked functions (e.g., `strncpy` or `strlcpy` over `strcpy`), avoid dangerous functions like `gets`, and include buffer size constants to prevent overflow. Enable compiler defenses (stack canaries, fortify source, DEP/NX) in any build configurations you suggest"*. By giving such instructions, the assistant might prefer safer standard library calls or even suggest modern C++ classes (`std::vector` instead of raw arrays) to reduce manual memory handling. It will also acknowledge when an operation is risky, possibly inserting comments like "// ensure no buffer overflow". [connordilgren2025a] [neilperry2022c]

- **Rust, Go, and Memory-Safe Languages:** If the project involves memory-safe languages (Rust, Go, Java, C#, etc.), you can note that the AI should leverage their safety features. *"In Rust code, avoid using `unsafe` blocks unless absolutely necessary and document any `unsafe` usage with justification".* Memory-safe-by-default languages enforce a lot at compile time, but you should still have the AI follow best practices of those ecosystems. For example, instruct: *"In any memory-safe language, prefer using safe library functions and types; don't circumvent their safety without cause".* If a language offers any tools to verify memory access, direct the AI assistant to use them while building or testing your code. For Example "*In go code, use the data race detector when building the application*". [openssf2023c]
- **Python and Dynamic Languages:** Python, JavaScript, and other high-level languages manage memory for you, but come with their own security pitfalls. In your instructions, emphasize things like avoiding exec/eval with untrusted input in Python and being careful with command execution. *"For Python, do not use `exec`/`eval` on user input and prefer safe APIs (e.g., use the `subprocess` module with `shell=False` to avoid shell injection)".* Additionally, mention type checking or the use of linters: *"Follow PEP 8 and use type hints, as this can catch misuse early".* For JavaScript/TypeScript, you might add: *"When generating Node.js code, use prepared statements for database queries (just like any other language) and encode any data that goes into HTML to prevent XSS".* These instructions incorporate known best practices (like those from OWASP cheat sheets) directly into the AI's behavior. [haoyan2025c]
- **Java/C# and Enterprise Languages:** In languages often used for large applications, you might focus on frameworks and configurations. *"For Java, when suggesting web code (e.g., using Spring), ensure to use built-in security annotations and avoid old, vulnerable libraries (e.g., use `BCryptPasswordEncoder` rather than writing a custom password hash)".* For C#, similarly: *"Use .NET's cryptography and identity libraries instead of custom solutions".* Also instruct about managing object deserialization (both Java and C# have had vulnerabilities in this area): *"Never suggest turning off security features like XML entity security or type checking during deserialization".* These language-specific notes guide the AI to incorporate the well-known secure patterns of each ecosystem. [neilperry2022d]

---

6. Referencing Security Standards and Frameworks

To strengthen your AI assistant's guidance, you should point it toward established security standards and frameworks. By referencing these in the instructions, the AI will be biased toward solutions that comply with widely accepted practices:

- **Use Industry Standards as Keywords:** Mention standards like **OWASP Top 10**, **OWASP ASVS**, **CWE/SANS Top 25**, or language-specific guidelines (Java's SEI CERT secure coding standards, for instance). For example: *"Code suggestions should adhere to OWASP Top 10 principles (e.g., avoid injection, enforce access control) and follow the OWASP ASVS requirements where applicable".* [arifulhaque2025c]
- **Reference SAFECode and Other Guides:** SAFECode (Software Assurance Forum for Excellence in Code) publishes **Fundamental Practices for Secure Software Development**, and including a note about it can be beneficial. *"Our project follows SAFECode's secure development practices – the AI should prioritize those (e.g., proper validation, authentication, cryptography usage per SAFECode guidance)".* This ties the assistant's suggestions to a comprehensive set of secure development principles. Similarly, you can reference the **Secure Software Development Lifecycle (SSDLC)** or standards like **ISO 27034** if your organization uses them, to align AI outputs with those processes. [neilperry2022e]

- **Compliance and Regulatory Security:** If you operate in an industry with specific regulations (like healthcare or finance), instruct the AI with those in mind. For instance: *"When generating code, consider compliance requirements (e.g., HIPAA privacy rules for medical data, PCI-DSS for credit card info) – do not output code that logs or transmits sensitive data in insecure ways"*. This ensures the AI's suggestions are compliant with external standards, for example, it might suggest encryption for personal data fields by default. [arifulhaque2025d]
- **Continuous Improvement Hooks:** Encourage practices that integrate with ongoing security processes. For example: *"Include comments or TODOs in code suggesting security reviews for complex logic, and note if any third-party component might need a future update or audit"*. This way, the AI not only writes code but also flags areas for human scrutiny or future action. If the AI outputs a piece of cryptography code, for instance, it might add a comment "// TODO: security review for cryptographic implementation" because you instructed it that critical code should not go un-reviewed. [anssibsi2024h] [fossa2023] [anssibsi2024d]
- **Integrate and Act on Automated Security Tooling:** Instruct the AI to proactively run automated security tools (e.g., SAST, DAST, SCA) during code generation and refinement. Example: "When writing or reviewing code, run or simulate the use of tools like CodeQL, Bandit, Semgrep, or OWASP Dependency-Check. Identify any flagged vulnerabilities or outdated dependencies and revise the code accordingly. Repeat this process until the code passes all simulated scans". This positions the AI not just as a code generator but as a security-aware assistant capable of iterating based on tool feedback. [anssibsi2024i] [frontiers2024a]

---

7. Appendix: Citations and References

[wiz2025a] "Rules files are an emerging pattern to allow you to provide standard guidance to AI coding assistants. You can use these rules to establish project, company, or developer specific context, preferences, or workflows" (Wiz Research - Rules Files for Safer Vibe Coding)

[ifip2021] "'harm' means negative consequences, especially when those consequences are significant and unjust." (International Federation for Information Processing - IFIP Code of Ethics and Professional Conduct)

[anssibsi2024a] "AI coding assistants are no substitute for experienced developers. An unrestrained use of the tools can have severe security implications." (ANSSI, BSI - AI Coding Assistants)

[markvero2025a] "on average, we could successfully execute security exploits on around half of the correct programs generated by each LLM; and in less popular backend frameworks, models further struggle to generate correct and secure applications (Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, Martin Vechev - Can LLMs Generate Correct and Secure Backends?)"

[shihchiehdai2025a] "One of the major concerns regarding LLM-generated code is its security. As code snippets generated by LLMs are increasingly incorporated into industrial-level software and systems, it is critical to ensure that LLM-generated code is free of vulnerabilities that could be exploited by attackers" (Shih-Chieh Dai, Jun Xu, Guanhong Tao - A Comprehensive Study of LLM Secure Code Generation)

[anssibsi2024b] "About 40% of the programs had security vulnerabilities... One cause of these security flaws is the use of outdated programs in the training data of the AI models, leading to the suggestion of outdated

and insecure best practices. This is evident, for example, in password encryption, where insecure methods such as MD5 or a single iteration of SHA-256 are still often used" (ANSSI, BSI - AI Coding Assistants)

[swaroopdora2025a] "Improve the prompt: The user should improve the prompt by indicating each and every aspect of the security parameters to derive the secure web application code from the LLMs" (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code: A Security-Centric Evaluation of Code Generation Capabilities in Large Language Models)

[haoyan2025a] "These reductions suggest that newer models can effectively leverage security hints to improve code safety" (Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, Ziyu Yao - Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation])

[swaroopdora2025b] "Injection attacks remain one of the most critical vulnerabilities in web applications (OWASP Top 10)... LLM-generated code must handle user input securely to prevent exploitation"(Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code)

[wiz2025b] "... systematic review shows that code injection (CWE-94) and OS command injection (CWE-78) are common in AI-generated code." (Wiz Research - Rules Files for Safer Vibe Coding)

[haoyan2025b] "All models frequently generate vulnerable code across diverse vulnerability types... All models tend to generate vulnerabilities for both common (top-25) and less common types" (Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, Ziyu Yao - Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation)

[hammondpearce2021a] "Overall, Copilot performed poorly in this CWE (CWE-522: Insufficiently Protected Credentials)... it frequently tried to generate code which was based on the insecure 'MD5' hashing algorithm" (Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri - Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions)

[neilperry2022a] "Participants with access to an AI assistant were ... significantly more likely to write an insecure solution ... also less likely to authenticate the final returned value" (Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh - Do Users Write More Insecure Code with AI Assistants?)

[swaroopdora2025c] "The analysis reveals critical vulnerabilities in authentication mechanisms, session management..." (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code)

[swaroopdora2025d] "Generic Error Messages: Ensuring error messages do not disclose username existence or password policies prevents attackers from gaining insights during brute-force attempts... Leaking system information through verbose error messages can provide attackers valuable insights into potential vulnerabilities within authentication systems" (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code)

[wiz2025c] "It's clear that currently AI-Generated code is not secure by default... (Wiz Research - Rules Files for Safer Vibe Coding)

[swaroopdora2025e] "All models require substantial improvements in authentication security, session management, error handling and HTTP security headers to align with current industry best practices and

established frameworks, such as the NIST cybersecurity guidelines " (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code)

[anssibsi2024c] "The generation of test cases is another area where coding assistants can offer support. They can automatically propose test cases and unit tests that also cover edge cases" (ANSSI, BSI - AI Coding Assistants)

[markvero2025b] "Even flagship models struggle to generate correct and secure application backends" (Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, Martin Vechev - Can LLMs Generate Correct and Secure Backends?)

[fossa2023] "It's important to pay particularly close attention to auto-generated comments... make sure you understand and agree with them... It's a good practice to consider implementing a code-tagging system to differentiate between AI- and human-created code." (FOSSA - 5 Ways to Reduce GitHub Copilot Security and Legal Risks)

[anssibsi2024d] "It might be beneficial to flag AI generated code blocks and to document the used AI tools. This might help security testing and it can also be a useful information for third-party auditors, e.g. in the context of a security certification process" (ANSSI, BSI - AI Coding Assistants)

[swaroopdora2025f] "Encryption safeguards sensitive data at rest and in transit. Weak encryption methods or lack of encryption can expose passwords, personal information, and financial data... If passwords are stored in plain text or hashed without salting, attackers with database access can easily decrypt credentials, leading to mass account breaches." (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code)

[josephspracklen2024a] "Package hallucination occurs when an LLM generates code that recommends or contains a reference to a package that does not actually exist. An adversary can exploit package hallucinations... by publishing a package to an open-source repository with the same name as the hallucinated or fictitious package and containing some malicious code/functionality" (Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs)

[openssf2023a] "Avoid... copy reused code from other packages... instead of using a package manager to automate identifying and updating reused packages... Use a package manager to manage it, one that records the specific version numbers" (OpenSSF - Secure Software Development Fundamentals)

[arifulhaque2025a] "LLM sometimes produces code that uses older ... libraries, which are not compliant with modern security. This increases the security risk... they are not updated with the latest security vulnerabilities and threats, leaving the generated code open to new attacks." (Ariful Haque, Sunzida Siddique, Md. Mahfuzur Rahman, Ahmed Rafi Hasan, Laxmi Rani Das, Marufa Kamal, Tasnim Masura, Kishor Datta Gupta - Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment)

[josephspracklen2024b] "LLMs cannot update themselves with new information after release and have no knowledge of the world beyond their training data cutoff date." (Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations)

[anssibsi2024e] "The creation of a Software Bill of Materials (SBOM) allows you to retrospectively understand whether vulnerable libraries were used and enables a targeted response if a vulnerability of

certain components becomes known." (ANSSI, BSI - AI Coding Assistants)

[josephspracklen2024c] "Public OSS repositories ... have implemented various measures, including ... software signing to mitigate the distribution of malicious packages... Self-refinement strategies... utilize the model itself to detect and refine potential hallucinations" (Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations)

[arifulhaque2025b] "LLM often lacks the deep contextual understanding needed to create secure code, resulting in sometimes irrelevant advice for specific security contexts" (Ariful Haque, Sunzida Siddique, Md. Mahfuzur Rahman, Ahmed Rafi Hasan, Laxmi Rani Das, Marufa Kamal, Tasnim Masura, Kishor Datta Gupta - Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment)

[shihchiehdai2025b] "... code directly executes the user-provided shell script without checking for malicious commands... It uses the system call os.system() to execute the copy command, which is passed as a string. If the original_location or new_location variables contain malicious input, this could lead to command injection" (Shih-Chieh Dai, Jun Xu, Guanhong Tao - A Comprehensive Study of LLM Secure Code Generation)

[swaroopdora2025g] "Although some models implement security measures to a limited extent, none fully align with industry best practices" (Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, Sandeep Kumar Shukla - The Hidden Risks of LLM-Generated Web Application Code: A Security-Centric Evaluation of Code Generation Capabilities in Large Language Models)

[neilperry2022b] "36% of participants with access to the AI assistant writing solutions that are vulnerable to SQL injections compared to 7% of the control group" (Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh - Do Users Write More Insecure Code with AI Assistants?)

[anssibsi2024f] "... detailed information about the infrastructure and resource configuration can be sensitive. The last part is particularly relevant if approaches like Infrastructure as Code (IaC) are used" (ANSSI, BSI - AI Coding Assistants)

[josephspracklen2024d] "The open nature of these repositories... makes them an attractive platform for malware distribution. For instance, a total of 245,000 malicious code packages were discovered in open-source software repositories in 2023 alone (Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations)

[hammondpearce2021b] "... we produce 89 different scenarios for Copilot to complete, producing 1,689 programs. Of these, we found approximately 40% to be vulnerable" (Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri - Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions)

[anssibsi2024g] "The use of uncontrolled access to coding assistants ... in the cloud should be prohibited for business software development." (ANSSI, BSI - AI Coding Assistants)

[openssf2023b] "Many projects apply a ... DevSecOps approach, which intentionally blend these processes together with development... having secure distribution, fielding, operations, and disposal is critical for software to be secure in the real world." (OpenSSF - Secure Software Development Fundamentals)

[connordilgren2025a] "The security vulnerabilities in LLM-generated correct code generally stem from missing conditional checks, incorrect memory allocations, or incorrect conditional checks" (Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, Yizheng Chen - Benchmarking LLMs for Secure Code Generation in Real-World Repositories)

[neilperry2022c] "... we observe that participants in the experiment group were significantly more likely to introduce integer overflow mistakes in their solutions" (Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh - Do Users Write More Insecure Code with AI Assistants?)

[openssf2023c] "Even when a language is memory safe by default, it is still vital to follow that language ecosystem's best practices and to use external tools to ensure safety not only within your code" (OpenSSF - The Memory Safety Continuum)

[haoyan2025c] "... we reveal that although LLMs are prone to generating insecure code, advanced models can benefit from vulnerability hints and fine-grained feedback to avoid or fix vulnerabilities." (Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, Ziyu Yao - Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation)

[neilperry2022d] "Often times, responses from the AI assistant use libraries that explicitly flag that they are insecure in the documentation for the library" (Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh - Do Users Write More Insecure Code with AI Assistants?)

[arifulhaque2025c] "Additionally, we can reduce security risks by using OWASP guidelines and using trusted libraries" (Ariful Haque, Sunzida Siddique, Md. Mahfuzur Rahman, Ahmed Rafi Hasan, Laxmi Rani Das, Marufa Kamal, Tasnim Masura, Kishor Datta Gupta - Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment)

[neilperry2022e] "We found that those who specified task instructions... generated more secure code." (Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh - Do Users Write More Insecure Code with AI Assistants?)

[arifulhaque2025d] "Compliance with regulations like GDPR and HIPAA is essential to minimize legal risks and uphold organizational reputation in sensitive environments" (Ariful Haque, Sunzida Siddique, Md. Mahfuzur Rahman, Ahmed Rafi Hasan, Laxmi Rani Das, Marufa Kamal, Tasnim Masura, Kishor Datta Gupta - Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment)

[anssibsi2024h] "Generated content, in particular source code, should generally be reviewed and understood by the developers... It might be beneficial to 'deconstruct' AI-generated code and the used prompts in public code reviews within the company. This allows knowledge and experiences to be shared across teams." (ANSSI, BSI - AI Coding Assistants)

[anssibsi2024i] "Automated vulnerability scanners or approaches like chatbots that critically question the generated source code ('source code critics') can reduce the risk" (ANSSI, BSI - AI Coding Assistants)

[frontiers2024a] "... post-processing the output ... has a measurable impact on code quality, and is LLM-agnostic... Presumably, non-LLM static analyzers or linters may be integrated as part of the code generation procedure to provide checks along the way and avoid producing code that is visibly incorrect or dangerous" (Frontiers - A systematic literature review on the impact of AI models on the security of code generation)

[josephspracklen2024e] “These 30 tests generated a total of 2.23 million packages in response to our prompts, of which 440,445 (19.7%) were determined to be hallucinations, including 205,474 unique non-existent packages (i.e. packages that do not exist in PyPI or npm repositories and were distinct entries in the hallucination count, irrespective of their multiple occurrences) (Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs)

[billtoulas2025a] “A new class of supply chain attacks named ‘slopsquatting’ has emerged from the increased use of generative AI tools for coding and the model’s tendency to “hallucinate” non-existent package names. The term slopsquatting was coined by security researcher Seth Larson as a spin on typosquatting, an attack method that tricks developers into installing malicious packages by using names that closely resemble popular libraries. Unlike typosquatting, slopsquatting doesn’t rely on misspellings. Instead, threat actors could create malicious packages on indexes like PyPI and npm named after ones commonly made up by AI models in coding examples” (Bill Toulas - AI-hallucinated code dependencies become new supply chain risk)

[josephspracklen2024f] “3 of the 4 models … proved to be highly adept in detecting their own hallucinations with detection accuracy above 75%. Table 2 displays the recall and precision values for this test, with similarly strong performance across the 3 proficient models. This phenomenon implies that each model’s specific error patterns are detectable by the same mechanisms that generate them, suggesting an inherent self-regulatory capability. The indication that these models have an implicit understanding of their own generative patterns that could be leveraged for self-improvement is an important finding for developing mitigation strategies.” (Spracklen - We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs)

[catherinetony2024a] “When using LLMs or LLM-powered tools like ChatGPT or Copilot… (1) Using RCI is preferable over the other techniques studied in this work, as RCI can largely improve the security of the generated code (up to an order of magnitude w.r.t weakness density) even when applied with just 2 iterations. This technique has stayed valuable over several versions of the LLM models, and, hence, there is an expectation that it will stay valid in the future as well. … (4) In cases where multi-step techniques like RCI are not feasible, using simple zero-shot prompting with templates similar to comprehensive prompts, that specify well-established secure coding standards, can provide comparable results in relation to more complex techniques.” (Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhiff, Riccardo Scandariato - Prompting Techniques for Secure Code Generation: A Systematic Investigation)

[catherinetony2024b] “Across all the examined LLMs, the persona/memetic proxy approach has led to the highest average number of security weaknesses among all the evaluated prompting techniques excluding the baseline prompt that does not include any security specifications.” (Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhiff, Riccardo Scandariato - Prompting Techniques for Secure Code Generation: A Systematic Investigation)

[connordilgren2025b] “The sec-generic and sec-specific prompts [which tell the system to act as an expert] … do not result in a consistent increase in secure-pass@1 scores [on a more realistic benchmark] The security-policy prompt provides the LLM with a stronger hint on how to generate secure code… This prompt also works better than sec-generic and sec-specific on SecRepoBench, but with a much smaller improvement of 1.6 percentage points on average [for full C/C++ programs, with larger increases on larger LLM models]” (Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, Yizheng Chen - Benchmarking LLMs for Secure Code Generation in Real-World Repositories)

Across all the examined LLMs, the persona/memetic proxy approach has led to the highest average number of security weaknesses among all the evaluated prompting techniques excluding the baseline prompt that does not include any security specifications." (Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhiff, Riccardo Scandariato - Prompting Techniques for Secure Code Generation: A Systematic Investigation)

8. Credit

This work's development was led by Avishay Balter.

Contributors include Avishay Balter (Microsoft), David A. Wheeler (The Linux Foundation), Mihai Maruseac (Google), Nell Shamrell-Harrington (Microsoft and Chair of Rust Foundation), Jason Goodsell (Microsoft), Roman Zhukov (Red Hat), and others.

This is a list of materials (documents, services, and so on) released by the Open Source Security Foundation (OpenSSF) Best Practices Working Group (WG).

For website terms of use, trademark policy, and other project policies please see https://lfprojects.org. This site is open source. View or contribute at GitHub.